

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (Bsc.IT Semester III)
Scala for D.S Practical

Practical - 27 , 28 & 29

Roll No. : S025	Name: Abhishek Pal
Class : SY-Bsc.IT	Batch : 00
Date of Assignment : 22-09-2026	Date/Time of Submission : 22-09-2026

27 . Implement one-hot encoding for a categorical column in a dataset.

CODE :

```
build.sbt  Practical27.scala x
1  object Practical27 extends App {
2    {
3      val data = List("Red", "Blue", "Green", "Red", "Blue")
4      for (value <- data)
5      {
6        val r = if (value == "Red") 1 else 0
7        val b = if (value == "Blue") 1 else 0
8        val g = if (value == "Green") 1 else 0
9        println(value + " -> Red:" + r + " Blue:" + b + " Green:" + g)
10     }
11 }
```

OUTPUT :

```
"C:\Program Files\Java\jdk1.8.0_181\bin
Red -> Red:1 Blue:0 Green:0
Blue -> Red:0 Blue:1 Green:0
Green -> Red:0 Blue:0 Green:1
Red -> Red:1 Blue:0 Green:0
Blue -> Red:0 Blue:1 Green:0

Process finished with exit code 0
```

28 . Compute the Euclidean distance between two Breeze vectors. Use it for nearest neighbor classification.

CODE :

```
build.sbt  Practical27.scala x
1  import breeze.linalg._
2
3  ▶ object Practical27 extends App {
4    {
5      val A = DenseVector(2.0, 3.0)
6      val B = DenseVector(5.0, 7.0)
7
8      val distance = norm(A - B)
9
10     println("Euclidean Distance: " + distance)
11
12     val C = DenseVector(2.5, 3.5)
13
14     val d1 = norm(C - A)
15     val d2 = norm(C - B)
16
17     if (d1 < d2)
18       println("Nearest Neighbor: A")
19     else
20       println("Nearest Neighbor: B")
21   }
```

OUTPUT :

```
"C:\Program Files\Java\jdk1.8.0_181\bin\jav
Euclidean Distance: 5.0
Nearest Neighbor: A

Process finished with exit code 0
```

29 . Create a simple Spark MLlib pipeline to classify data. Use logistic regression or decision trees.

CODE :

```
build.sbt  Practical27.scala ×
1  import org.apache.spark.sql.SparkSession
2  import org.apache.spark.ml.feature.VectorAssembler
3  import org.apache.spark.ml.classification.LogisticRegression
4
5  ▶ object Practical27 extends App { new *
6
7      val spark = SparkSession.builder()
8          .appName("Classification")
9          .master("local[*]")
10         .getOrCreate()
11
12     import spark.implicits._
13
14     val data = Seq(
15         (1.0, 0.0),
16         (2.0, 0.0),
17         (3.0, 0.0),
18         (7.0, 1.0),
19         (8.0, 1.0),
20         (9.0, 1.0)
21     ).toDF("marks", "label")
22
23     val a = new VectorAssembler()
24         .setInputCols(Array("marks"))
25         .setOutputCol("features")
26
```

```

val d = a.transform(data)

val model = new LogisticRegression()
    .setLabelCol("label")
    .setFeaturesCol("features")

val result = model.fit(d)

val prediction = result.transform(d)

prediction.select("marks", "label", "prediction").show()

spark.stop()
}

```

OUTPUT :

```

20/07/22 10:14:27 INFO SparkUI:
+-----+-----+-----+
|marks|label|prediction|
+-----+-----+-----+
|  1.0|  0.0|        0.0|
|  2.0|  0.0|        0.0|
|  3.0|  0.0|        0.0|
|  7.0|  1.0|        1.0|
|  8.0|  1.0|        1.0|
|  9.0|  1.0|        1.0|
+-----+-----+-----+

```